Лекция 1. Разведывательный анализ данных

Введение

Анализ данных начинается не с построения моделей машинного обучения, а с понимания самих данных. Прежде чем обучать модель, необходимо ответить на целый ряд вопросов: что представляют собой данные, какого они качества, есть ли пропуски, выбросы и дубликаты, какие признаки информативны, какие закономерности видны уже на этапе первичного анализа. Этот этап называется разведывательным анализом данных (Exploratory Data Analysis, EDA). В курсе «Анализ данных в Python» EDA опирается на библиотеки NumPy, Pandas и SciPy [1].

Исторически термин EDA ввёл американский статистик Джон Тьюки в 1977 году. Его идея состояла в том, что анализ данных должен начинаться не с проверки готовых гипотез, а с исследования самих данных — их структуры, распределений, взаимосвязей и аномалий [3]. Тьюки противопоставлял разведывательный анализ подтверждающему (confirmatory data analysis), при котором гипотезы формулируются заранее и затем проверяются. В современном анализе данных EDA стал стандартным первым этапом любого проекта — от простой обработки таблицы до сложных систем машинного обучения [4].

Разведывательный анализ решает несколько задач. Во-первых, он позволяет понять структуру данных: сколько наблюдений, какие признаки, какого они типа. Во-вторых, он выявляет проблемы качества: пропуски, дубликаты, выбросы, некорректные значения. В-третьих, он помогает обнаружить закономерности: распределения, взаимосвязи, группировки. В-четвёртых, он формирует гипотезы для дальнейшего анализа и построения моделей [2]. Без EDA невозможно грамотно подготовить данные, а без подготовки данных невозможно построить качественную модель.

Библиотека NumPy

Основой для работы с данными в Python служит библиотека NumPy — базовая библиотека для работы с многомерными массивами. Она обеспечивает быстрые поэлементные вычисления, математические и статистические функции и является фундаментом для Pandas, SciPy и Scikit-learn. Основной объект NumPy — N-мерный массив ndarray. В отличие от списка Python, массив NumPy хранит элементы одного типа, поддерживает векторные операции и работает значительно быстрее.

Создание массивов в NumPy осуществляется несколькими способами. Функция np.array() создаёт массив из списка или кортежа. Функция np.zeros() создаёт массив из нулей заданной формы. Функция np.ones() создаёт массив из единиц. Функция np.arange() создаёт последовательность чисел с заданным шагом. Функция np.linspace() создаёт равномерно распределённые значения в заданном диапазоне. Функция np.random.rand() создаёт массив случайных чисел из равномерного распределения, а np.random.randn() — из нормального.

```
import numpy as np
a = np.array([1, 2, 3, 4, 5])
b = np.zeros((3, 3))
c = np.ones((2, 4))
d = np.arange(0, 10, 2)
e = np.linspace(0, 1, 5)
f = np.random.rand(3, 3)
g = np.random.randn(100)

print('a = ', a)
print('b = ',b)
print('c = ',c)
```

```
print('d = ',d)
print('e = ',e)
print('f = ',f)
print('g = ',g)
```

```
a =  [1 2 3 4 5]
b =  [[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
c =  [[1. 1. 1. 1.]
 [1. 1. 1. 1.]]
d =  [0 2 4 6 8]
e =  [0.   0.25 0.5  0.75 1.  ]
f =  [[0.45685931 0.643974   0.13991701]
 [0.10734662 0.2072547  0.83391637]
 [0.75777844 0.91365361 0.2109302 ]]
g =  [ 0.08770005 -0.19222665  1.92567157  0.59351423  0.06608489 -1.62366845
 -0.63501552 -3.1609752  -0.79759264 -1.87827548  0.30425684  0.96204468
  0.1020285   1.35698907 -0.4814818  -1.19166995 -1.15623257  0.36845323
 -0.06139716  0.97432352 -0.64220209 -0.15042357  0.70279817  0.82717412
 -1.22512536  1.69767458  2.71875742  1.49171269 -1.22421314 -0.55253486
 -0.61474831  2.58137611  0.65491988  0.11952858  0.45824709 -0.08389687
 -1.30613188 -0.73555685  1.68311505  0.6842772  -0.8373351  -1.17481687
 -0.47631629 -1.07955166 -0.27195755 -0.65466157 -1.03429154 -1.65726796
 -0.55062117 -0.31886029 -0.44579908 -0.79623321 -0.3113803   1.10630896
 -0.3729828  -0.9257179  -0.15799242 -1.78703418  1.47206889 -0.06058993
 -1.73957629  0.22207471  1.89540579 -0.74085958 -1.49470114  0.72194819
  0.15262819  0.59142276 -2.55682806  0.89787957  0.92919427 -0.57194095
  0.47416197  1.53086927 -0.44874947  0.17845704 -1.19884296  0.74893571
  1.48937512 -2.60874733  0.21076969 -0.28895636  0.09807383 -0.22636921
 -1.38253477  0.52296357  0.45575045  0.16151791  2.36312169 -0.32167318
 -0.88366311 -0.40701677  0.16805253  0.29537507  0.12441256 -2.40654973
```

Массивы NumPy поддерживают разнообразные операции. Поэлементные операции выполняются без циклов: сложение, вычитание, умножение, деление, возведение в степень. Агрегатные функции позволяют вычислить сумму, среднее, медиану, стандартное отклонение, минимум, максимум.

Функции np.mean(), np.median(), np.std(), np.sum(), np.min(), np.max() работаю т как с одномерными, так и с многомерными массивами, а параметр axis позволяет указать направление агрегации.

```
a = np.array([1, 2, 3, 4, 5])
print('a.mean() = ',a.mean())      # 3.0
print('a.std() = ',a.std())       # 1.41
```

```
print('a.sum()= ', a.sum())       # 15
print('a.max()= ', a.max())  # 1 5

b = np.array([[1, 2, 3], [4, 5, 6]])
print('b.mean(axis=0) = ', b.mean(axis=0))  # [2.5 3.5 4.5]
print('b.mean(axis=1) = ', b.mean(axis=1))  # [2. 5.]
```

Индексация и срезы в NumPy работают аналогично спискам Python, но с расширенными возможностями. Можно использовать отрицательные индексы, срезы с шагом, булевы маски. Булевы маски позволяют выбирать элементы, удовлетворяющие условию: a[a > 3] вернёт все элементы больше трёх. Это очень удобно для фильтрации данных.

```
a = np.array([1, 2, 3, 4, 5])
print(a[0])       # 1
print(a[-1])      # 5
print(a[1:4])     # [2 3 4]
print(a[::2])     # [1 3 5]
print(a[a > 3])   # [4 5]
```

NumPy также поддерживает изменение формы массива: reshape() позволяет преобразовать массив в другую форму, flatten() — превратить многомерный массив в одномерный, transpose() — транспонировать. Объединение массивов осуществляется функциями concatenate(), stack(), hstack(), vstack(). Разделение — функциями split(), hsplit(), vsplit().

```
a = np.arange(10)
b = a.reshape(2, 5)
c = b.flatten()
d = b.T
e = np.concatenate([a, a])
```

```
f = np.vstack([a, a])
```

NumPy также предоставляет функции линейной алгебры: умножение матриц (np.dot(), @), обратная матрица (np.linalg.inv()), определитель (np.linalg.det()), собственные значения (np.linalg.eig()), решение систем линейных уравнений (np.linalg.solve()). Эти функции используются в анализе данных, машинном обучении, обработке сигналов.

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
print(A @ B)
print(np.linalg.det(A))
print(np.linalg.inv(A))
[[19 22]
 [43 50]]
-2.0000000000000004
[[-2.   1. ]
 [ 1.5 -0.5]]
```

Библиотека Pandas

Библиотека Pandas предназначена для работы с табличными данными. Её основные структуры — Series (одномерный массив с индексами) и DataFrame (двумерная таблица с именованными столбцами). Series можно представить как столбец таблицы, а DataFrame — как саму таблицу. DataFrame позволяет загружать данные из CSV и Excel, фильтровать строки и столбцы, группировать и агрегировать данные, объединять таблицы, а также обрабатывать пропуски.

Загрузка данных осуществляется функциями pd.read_csv(), pd.read_excel(), pd.read_json(), pd.read_sql(). После

загрузки данные попадают в DataFrame, с которым можно работать. Просмотр первых строк осуществляется методом head(), последних — tail(). Информация о данных доступна через info(): типы столбцов, количество непустых значений, объём памяти. Описательные статистики вычисляются методом describe(): среднее, медиана, стандартное отклонение, квартили, минимум и максимум.

```
import pandas as pd

df = pd.read_csv('data.csv')
print(df.head())
print(df.info())
print(df.describe())
print(df.isnull().sum())
print(df.duplicated().sum())
```

Фильтрация данных осуществляется с помощью булевых условий. Например, df[df['age'] > 30] вернёт все строки, где возраст больше 30. Можно комбинировать условия через & (и), | (или), ~ (не). Выбор столбцов осуществляется через двойные квадратные скобки: df[['age', 'fare']]. Выбор строк по индексу — через loc (по метке) и iloc (по позиции).

```
df[df['age'] > 30]
df[(df['age'] > 30) & (df['sex'] == 'female')]
df[['age', 'fare', 'survived']]
df.loc[0:5, 'age']
df.iloc[0:5, 0:3]
```

	survived	pclass	sex
0	0	3	male
1	1	1	female
2	1	3	female
3	1	1	female
4	0	3	male

Группировка и агрегация данных осуществляется методом groupby(). Он позволяет сгруппировать данные по одному или нескольким признакам и вычислить агрегатные функции для каждой группы. Например, df.groupby('sex')['survived'].mean() вернёт среднюю выживаемость для каждого пола. Можно применять несколько агрегатных функций одновременно через agg().

```
df.groupby('sex')['survived'].mean()
df.groupby('pclass')['fare'].agg(['mean', 'median', 'std'])
df.groupby(['sex', 'pclass'])['survived'].mean()
```

		survived
sex	**pclass**	
female	**1**	0.968085
	2	0.921053
	3	0.500000
male	**1**	0.368852
	2	0.157407
	3	0.135447

Объединение таблиц осуществляется функциями merge(), join(), concat(). merge() объединяет таблицы по общему столбцу (аналог SQL JOIN), join() — по индексу, concat() — по строкам или

столбцам. Виды объединения: inner (пересечение), outer (объединение), left (все строки левой таблицы), right (все строки правой таблицы).

```
df1.merge(df2, on='id', how='inner')
df1.merge(df2, on='id', how='left')
pd.concat([df1, df2], axis=0)
pd.concat([df1, df2], axis=1)
```

Pandas также поддерживает мультииндексы (MultiIndex), сводные таблицы (pivot tables), операции stack и unstack. Мультииндексы позволяют работать с иерархическими данными. Сводные таблицы позволяют агрегировать данные по двум и более признакам. Операции stack и unstack преобразуют данные между «широким» и «длинным» форматами.

```
df.pivot_table(values='survived', index='sex', columns='pclass', aggfunc='mean')
df.stack()
df.unstack()
```

Библиотека SciPy

Библиотека SciPy применяется для научных вычислений, и в анализе данных она используется для описательной статистики, работы с вероятностными распределениями и проверки статистических гипотез. Модуль scipy.stats предоставляет функции для вычисления статистик, проверки нормальности распределения, корреляционного анализа, сравнения групп.

Описательная статистика включает вычисление среднего, медианы, моды, дисперсии, стандартного отклонения, асимметрии, эксцесса.

Функции stats.describe(), stats.normaltest(), stats.shapiro() позволяют проверить нормальность распределения. Корреляционный анализ осуществляется функциями stats.pearsonr() (коэффициент Пирсона) и stats.spearmanr() (коэффициент Спирмена). Сравнение групп — функциями stats.ttest_ind() (t-критерий для независимых выборок), stats.ttest_rel() (t-критерий для связанных выборок), stats.mannwhitneyu() (непараметрический критерий), stats.kruskal() (сравнение более двух групп).

```
from scipy import stats
stat, p = stats.ttest_ind(group1, group2)
print(f'p-value = {p:.4f}')
r, p = stats.pearsonr(x, y)
print(f'Correlation = {r:.3f}, p-value = {p:.4f}')
stat, p = stats.shapiro(data)
print(f'Shapiro test p-value = {p:.4f}')
```

Загрузка и первичный анализ данных

Первичный анализ данных включает несколько обязательных шагов. Первый шаг — загрузка данных. Данные могут храниться в разных форматах: CSV, Excel, JSON, SQL-базы, API. Pandas позволяет загружать данные из большинства этих источников. Второй шаг — просмотр первых строк (df.head()), который даёт первое представление о структуре таблицы. Третий шаг — получение информации о данных (df.info()): типы столбцов, количество непустых значений, объём памяти. Четвёртый шаг — расчёт описательных статистик (df.describe()): среднее, медиана, стандартное отклонение, квартили, минимум и максимум. Пятый шаг — проверка пропусков (df.isnull().sum()). Шестой шаг — проверка дубликатов (df.duplicated().sum()).

Эти шаги дают первое представление о качестве данных и возможных проблемах. Например, если в столбце много пропусков, это может повлиять на выбор метода обработки. Если в данных есть дубликаты, их нужно удалить, чтобы не искажать статистику. Если типы данных не соответствуют ожидаемым (например, число хранится как строка), это нужно исправить.

```
import pandas as pd
import seaborn as sns
df = sns.load_dataset('titanic')
print(df.head())
print(df.info())
print(df.describe())
print(df.isnull().sum())
print(df.duplicated().sum())
```

```
3         1      1  female  35.0      1      0  53.1000        S  First
4         0      3    male  35.0      0      0   8.0500        S  Third

     who  adult_male deck  embark_town alive  alone
0    man        True  NaN  Southampton    no  False
1  woman       False    C    Cherbourg   yes  False
2  woman       False  NaN  Southampton   yes   True
3  woman       False    C  Southampton   yes  False
4    man        True  NaN  Southampton    no   True
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   survived     891 non-null    int64
 1   pclass       891 non-null    int64
 2   sex          891 non-null    object
 3   age          714 non-null    float64
 4   sibsp        891 non-null    int64
 5   parch        891 non-null    int64
 6   fare         891 non-null    float64
 7   embarked     889 non-null    object
 8   class        891 non-null    category
 9   who          891 non-null    object
 10  adult_male   891 non-null    bool
 11  deck         203 non-null    category
 12  embark_town  889 non-null    object
 13  alive        891 non-null    object
 14  alone        891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.7+ KB
None
         survived      pclass         age       sibsp       parch        fare
count  891.000000  891.000000  714.000000  891.000000  891.000000  891.000000
mean     0.383838    2.308642   29.699118    0.523008    0.381594   32.204208
std      0.486592    0.836071   14.526497    1.102743    0.806057   49.693429
min      0.000000    1.000000    0.420000    0.000000    0.000000    0.000000
25%      0.000000    2.000000   20.125000    0.000000    0.000000    7.910400
50%      0.000000    3.000000   28.000000    0.000000    0.000000   14.454200
75%      1.000000    3.000000   38.000000    1.000000    0.000000   31.000000
max      1.000000    3.000000   80.000000    8.000000    6.000000  512.329200
survived         0
pclass           0
sex              0
age            177
sibsp            0
parch            0
fare             0
embarked         2
class            0
who              0
adult male       0
```

Рисунок 1 – Данные

Обработка пропущенных значений

Пропущенные значения — распространённая проблема реальных данных. Они могут возникать по разным причинам: ошибки при вводе, отсутствие информации, сбои при передаче данных. Существует несколько способов обработки пропусков. Удаление строк (dropna) применяется, если пропусков мало и они не несут важной информации. Заполнение константой используется, если пропуск означает «нет значения» (например, отсутствие категории). Заполнение средним или медианой применяется для числовых

признаков; медиана предпочтительнее, так как она устойчива к выбросам. Заполнение модой применяется для категориальных признаков. Интерполяция используется для временных рядов, где значения меняются постепенно.

Например, заполнить пропуски в столбце возраста можно медианой:

df['age'] = df['age'].fillna(df['age'].median()),

а в категориальном столбце — модой:

df['embarked'] = df['embarked'].fillna(df['embarked'].mode()[0]).

Важно обосновывать выбор метода: медиана устойчива к выбросам, мода подходит для категорий. Иногда полезно создать отдельный признак-индикатор пропуска, чтобы модель могла учитывать сам факт отсутствия значения.

```
df['age'] = df['age'].fillna(df['age'].median())
df['embarked'] = df['embarked'].fillna(df['embarked'].mode()[0])
df = df.dropna(subset=['fare'])
df['age_missing'] = df['age'].isnull().astype(int)
```

Обработка дубликатов и выбросов

Дубликаты строк удаляются методом drop_duplicates(). Дубликаты могут возникать при объединении данных из разных источников, при повторной загрузке данных, при ошибках ввода. Их наличие искажает статистику и может привести к переобучению модели. Поэтому перед анализом важно проверить данные на дубликаты и удалить их.

```
df = df.drop_duplicates()
print(df.duplicated().sum())
```

Выбросы — значения, сильно отклоняющиеся от основной массы данных. Они могут быть как ошибками (например, возраст 200 лет), так и реальными аномалиями (например, крупная транзакция). Методы обнаружения выбросов: правило трёх сигм (значения за пределами среднего ± 3 стандартных отклонения), метод межквартильного размаха (IQR) — значения за пределами Q1 − 1.5·IQR и Q3 + 1.5·IQR, визуальные методы — box-plot. Способы обработки выбросов: удаление, замена статистиками (например, медианой), преобразование (логарифмирование, winsorization). Выбор метода зависит от природы выбросов и задачи.

```
Q1 = df['fare'].quantile(0.25)
Q3 = df['fare'].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
df = df[(df['fare'] >= lower) & (df['fare'] <= upper)]
```

Кодирование категориальных признаков

Алгоритмы машинного обучения работают с числами, поэтому категориальные признаки необходимо закодировать. Основные методы кодирования: One-Hot Encoding, при котором каждая категория превращается в отдельный бинарный признак (подходит для номинальных признаков); Label Encoding, при котором каждой категории присваивается числовой код (подходит для порядковых признаков); Target Encoding, при котором категория заменяется средним значением целевой переменной (применяется при большом числе категорий). Существуют и другие методы: Frequency Encoding (замена категории на её частоту), Binary Encoding (бинарное представление категорий), Hashing (хеширование категорий).

```
from sklearn.preprocessing import OneHotEncoder, LabelEncoder
encoder = OneHotEncoder(drop='first', sparse_output=False)
X_encoded = encoder.fit_transform(df[['sex', 'embarked']])
le = LabelEncoder()
df['sex_encoded'] = le.fit_transform(df['sex'])
```

Важно помнить, что One-Hot создаёт много признаков при большом числе категорий (так называемое «проклятие размерности»), Label создаёт ложный порядок для номинальных признаков, а Target Encoding склонен к переобучению. Поэтому выбор метода зависит от типа признака, числа категорий и модели, которая будет использоваться.

Масштабирование числовых признаков

Числовые признаки могут иметь разный масштаб, что мешает алгоритмам, чувствительным к расстояниям и градиентам. Например, возраст измеряется в годах (0–100), а зарплата — в рублях (0–1 000 000). Если не масштабировать признаки, алгоритмы, основанные на расстояниях (KNN, SVM), будут отдавать предпочтение признакам с большим масштабом. Для приведения признаков к единому масштабу применяются StandardScaler (преобразует данные к среднему 0 и дисперсии 1), MinMaxScaler (сжимает данные в диапазон [0, 1]) и RobustScaler (использует медиану и межквартильный размах, устойчив к выбросам). Выбор метода зависит от свойств данных: при выбросах предпочтителен RobustScaler, при нормальном распределении — StandardScaler.

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df[['age', 'fare']])
```

```
minmax = MinMaxScaler()
X_minmax = minmax.fit_transform(df[['age', 'fare']])
robust = RobustScaler()
X_robust = robust.fit_transform(df[['age', 'fare']])
```

Масштабирование особенно важно для алгоритмов, чувствительных к масштабу: логистическая регрессия, метод опорных векторов, K-ближайших соседей, нейронные сети. Для деревьев решений и случайного леса масштабирование не требуется, так как они работают с пороговыми значениями.

Разведывательный анализ данных (EDA)

Разведывательный анализ данных — это исследовательский этап, на котором аналитик изучает распределения признаков, выявляет взаимосвязи, проверяет гипотезы и формулирует выводы. Основные шаги EDA включают расчёт описательных статистик, анализ распределений с помощью гистограмм и box-plot, анализ взаимосвязей через корреляцию и диаграммы рассеяния, анализ категориальных признаков через столбчатые диаграммы и группировки, а также проверку гипотез с помощью статистических тестов.

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.histplot(df['age'], kde=True)
plt.title('Распределение возраста')
plt.show()

sns.boxplot(x='pclass', y='fare', data=df)
plt.title('Стоимость билета по классам')
```

```
plt.show()

sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title('Тепловая карта корреляций')
plt.show()

sns.countplot(x='sex', hue='survived', data=df)
plt.title('Выживаемость по полу')
plt.show()
```

Корреляционный анализ позволяет выявить линейные взаимосвязи между признаками; диаграммы рассеяния — нелинейные; box-plot — различия между группами. EDA — это итеративный процесс: на каждом шаге аналитик задаёт вопросы к данным и ищет на них ответы. Например, увидев, что выживаемость женщин выше, чем мужчин, можно задать вопрос: связано ли это с классом каюты? Увидев, что распределение возраста имеет два пика, можно задать вопрос: связаны ли эти пики с полом или классом? Такие вопросы помогают глубже понять данные и сформулировать гипотезы для дальнейшего анализа.

Выводы по лекции

В ходе лекции мы рассмотрели основные библиотеки Python для анализа данных — NumPy, Pandas и SciPy, загрузку и первичный анализ данных, обработку пропусков, дубликатов и выбросов, кодирование категориальных признаков, масштабирование числовых признаков и разведывательный анализ данных как этап исследования. Ключевая идея лекции состоит в том, что качество модели машинного обучения напрямую

зависит от качества подготовки данных, а EDA — это не формальность, а важнейший этап, на котором формируется понимание данных.

Контрольные вопросы:

1. Какие библиотеки Python используются для анализа данных и в чём их основное назначение?
2. Что такое DataFrame и как его использовать в Pandas?
3. Какие основные методы анализа данных можно использовать с помощью Pandas?
4. Как обрабатывать пропущенные значения и дубликаты в DataFrame?
5. Что такое разведывательный анализ данных (EDA) и какие этапы он включает?

Список источников:

1. Анализ данных : учебник для вузов / В. С. Мхитарян [и др.] ; под ред. В. С. Мхитаряна. — Москва : Юрайт, 2022. — 490 с.
2. Вьюненко, Л. Ф. Анализ данных : учебник для вузов / Л. Ф. Вьюненко, Ю. Н. Гагарина, Н. Ю. Гагарина. — Москва : Юрайт, 2024. — 297 с.
3. Плас, Дж. Вандер. Python для сложных задач: наука о данных и машинное обучение. — Санкт-Петербург : Питер, 2022. — 576 с.
4. Грас, Дж. Data Science. Наука о данных с нуля. — Санкт-Петербург : БХВ-Петербург, 2021. — 336 с.